华清远见
HQYJ.COM

PyTorch的tensor

韩雁泽

华清远见|元宇宙实验中心
yyzlab.com.cn

目录

CONTENTS

01. PyTorch是什么

02. tensor是什么

03. tensor的存储机制

04. tensor的连续性

华清远见 HQYJ.COM

华清远见|元宇宙实验中心 yyulab.com.cn

01 PyTorch是什么

华清远见
HQYJ.COM
华清远见|元宇宙实验中心
yyzlab.com.cn

PyTorch是什么？

华清远见 HQYJ.COM

概念：

PyTorch 是一个开源的深度学习框架，由Meta公司（原名：Facebook）的人工智能研究团队开发和维护。它提供了一个灵活、动态的计算图计算模型，使得在深度学习领域进行实验和开发变得更加简单和直观。

华清远见嵌入式实验中心 yyzlab.com.cn

PyTorch特点

华清远见 HQYJ.COM

动态计算图

PyTorch 使用动态计算图，这意味着计算图是在运行时构建的，而不是在编译时静态定义的。

自动求导（微分）

PyTorch 提供了自动求导机制，称为 Autograd。它能够自动计算张量的梯度，这对于训练神经网络和其他深度学习模型非常有用。

丰富的神经网络库

PyTorch 提供了丰富的神经网络库，包括各种各样的层、损失函数、优化器等。这些库使得构建和训练神经网络变得更加容易。

支持GPU加速

PyTorch 充分利用了 GPU 的并行计算能力，能够在 GPU 上高效地进行计算，加速模型训练过程。

华清远见元宇宙实验中心 yyzlab.com.cn

02 tensor是什么

华清远见
HQYJ.COM
华清远见|元宇宙实验中心
yyzlab.com.cn

tensor

tensor是一种多维数组，类似于NumPy的ndarray。它是PyTorch中最基本的数据结构，用于存储和操作数据。tensor可以是标量、向量、矩阵或更高维度的数组，可以包含整数、浮点数或其他数据类型的元素。

PyTorch的tensor与NumPy的ndarray非常相似，但在设计和功能上有一些不同之处。主要的区别包括：

GPU加速、自动求导、动态计算图。

```
Scalar Tensor:
tensor(3.1400)

Vector Tensor:
tensor([1, 2, 3, 4, 5])

Matrix Tensor:
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
```

```
Tensor Tensor:
tensor([[[ 1,  2],
         [ 3,  4]],

        [[ 5,  6],
         [ 7,  8]],

        [[ 9, 10],
         [11, 12]]])
```

华清远见元宇宙实验中心
yyzlab.com.cn

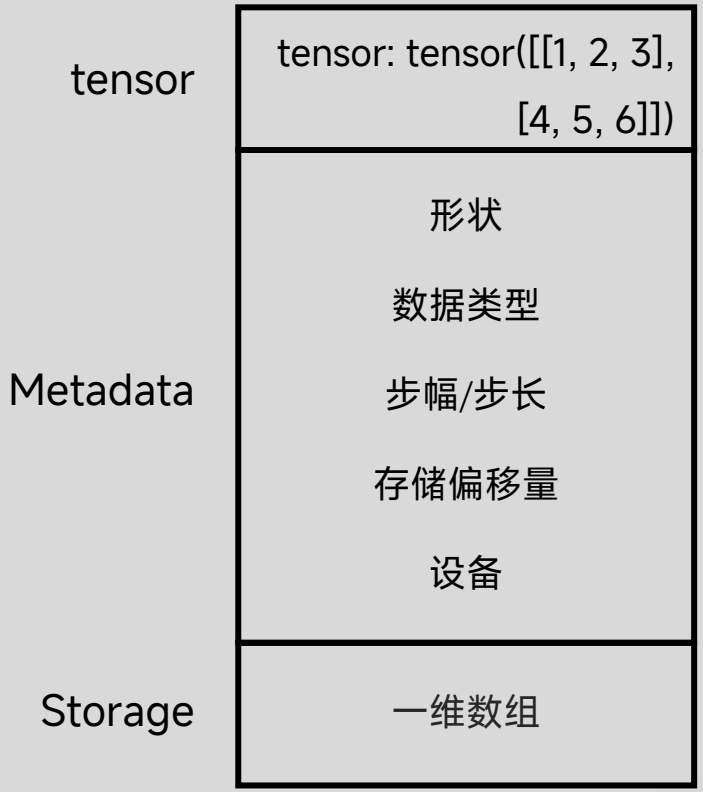
03 tensor的存储机制

华清远见
HQYJ.COM
华清远见|元宇宙实验中心
yyzlab.com.cn

tensor

在 PyTorch 中，tensor 包含了两个部分，即 storage 和 metadata。Storage（存储）：存储是 tensor 中包含的实际数据的底层缓冲区，它是一维数组，存储了 tensor 的元素值。不同 tensor 可能共享相同的存储，即使它们具有不同的形状和步幅。存储是一块连续的内存区域，实际上存储了 tensor 中的数据。

Metadata（元数据）：元数据是 tensor 的描述性信息，包括 tensor 的形状、数据类型、步幅、存储偏移量、设备等。元数据提供了关于 tensor 的结构和属性的信息，但并不包含 tensor 中的实际数据。元数据允许 PyTorch 知道如何正确地解释存储中的数据以及如何访问它们。



华清远见 HQYJ.COM
华清远见元宇宙实验中心 yyzlab.com.cn

数据类型

华清远见 HQYJ.COM

数据类型	具体类型	释义	范围
整数类型	torch.int8	8位有符号整数	-128 到 127
	torch.int16	16位有符号整数	-32768 到 32767
	torch.int32/torch.int	32位有符号整数	-2147483648 到 2147483647
	torch.int64	64位有符号整数	-9223372036854775808 到 9223372036854775807
无符号整数类型	torch.uint8	8位无符号整数	0 到 255
	torch.uint16	16位无符号整数	0 到 65535
	torch.uint32	32位无符号整数	0 到 4294967295
	torch.uint64	64位无符号整数	0 到 18446744073709551615

华清远见研究中心 yjzlab.com.cn

数据类型

华清远见 HQYJ.COM

数据类型	具体类型	释义	范围
浮点数类型	torch.float32/ torch.float	32位浮点数，通常称为单精度浮点数	符号位：1位，用于表示数值的正负。 指数部分：8位，用于表示数值的指数部分。 小数部分：23位，用于表示数值的小数部分。
	torch.float64/ torch.double	64位浮点数，通常称为双精度浮点数	符号位：1位，用于表示数值的正负。 指数部分：11位，用于表示数值的指数部分。 小数部分：52位，用于表示数值的小数部分。
	torch.float16/ torch.half	16位浮点数，通常称为半精度浮点数	符号位：1位，用于表示数值的正负。 指数部分：5位，用于表示数值的指数部分。 小数部分：10位，用于表示数值的小数部分。

举例：

0.75的float16的十进制是多少？

0011101000000000的float16的二进制是多少？

华清远见 yyzlab.com.cn

答案

首先，我们需要了解float16格式的结构：

在float16中，符号位占1位，指数部分占5位，小数部分（即尾数）占10位。

对于给出的float16值：0011101000000000

- 符号位：0（表示这是一个正数）
- 指数部分：01110（二进制），转换为十进制是$2^3 + 2^2 + 2^1 = 8 + 4 + 2 = 14$
- 小数部分（尾数）：1000000000（二进制）

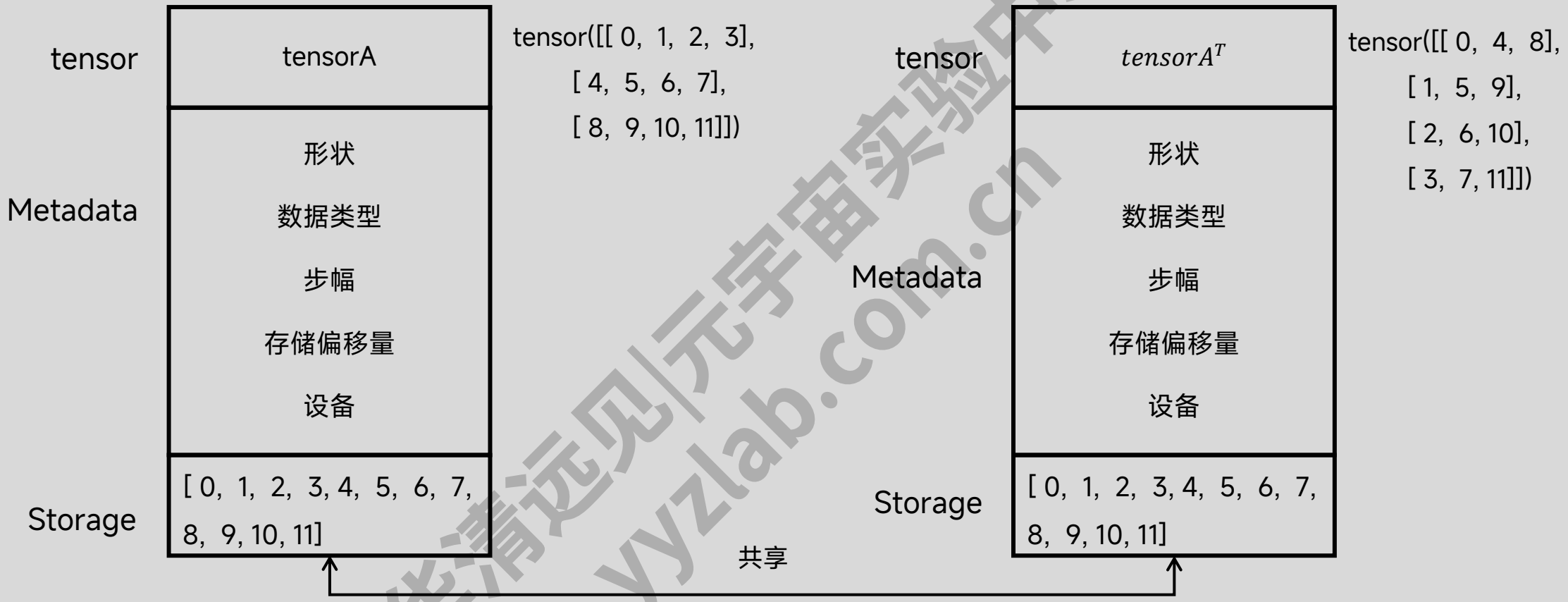
接下来，将指数部分转换为实际的指数值。在float16中，指数的偏移量是15，因为最大为$2^{5-1} - 1 = 15$，所以实际的指数是14-15=-1。

尾数部分需要加上隐含的前导1（因为在规格化的浮点数中，前导的1是隐含的），先将小数部分转成十进制：$2^{-1} = 0.5$，加上前导1，所以实际尾数得到1.5。

最终计算方法：$(-1)^{符号位} \times 2^{-1} \times 1.5 = 0.75$。

华清远见元宇宙实验中心
yyzlab.com.cn

Storage存储与共享



华清远见元宇宙实验中心
yyzlab.com.cn

对张量进行操作时（比如调整形状、转置等），它不会创建一个新的张量，而是返回一个指向相同数据的视图。

即PyTorch通常会共享相同的存储对象，以节省内存和提高效率。

Storage存储的具体过程

在PyTorch中，张量的存储是通过torch.Storage类来管理的。张量的值被分配在连续的内存块中，这些内存块是大小可变的一维数组，可以包含不同类型的数据，如float或int32。

具体来说，当我们创建一个张量时，PyTorch会根据我们提供的数据和指定的数据类型来分配一块连续的内存空间。这块内存空间由torch.Storage对象管理，而张量本身则提供了一种视图，让我们可以通过多维索引来访问这些数据。

此外，PyTorch还提供了一系列的函数和方法来操作张量，包括改变形状、获取元素、拼接和拆分等。这些操作通常不会改变底层的存储，而是返回一个新的张量视图，这个视图指向相同的数据但可能有不同的形状或索引方式。

总的来说，张量的存储实现是PyTorch能够高效进行张量运算的关键。通过管理一块连续的内存空间，并提供了丰富的操作方法，使得用户可以方便地对多维数组进行各种计算和变换。

华清远见 HQYJ.COM
华清远见云计算实验中心 yvzlab.com.cn

tensor的步长

华清远见 HQYJ.COM

步长指的是在每个维度上移动一个元素时在底层存储中需要跨越的元素数。

```
a = torch.arange(12).reshape( *shape: 3, 4)
print(a)
print(a.stride())
```

第一个数字4表示在矩阵的行（第一个维度）上移动一个元素时需要跨越的存储单元数。因为矩阵的每行包含4个元素，所以每次沿着行移动一个元素需要跨越4个存储单元。

第二个数字1表示在矩阵的列（第二个维度）上移动一个元素时需要跨越的存储单元数。因为矩阵的列数是1，所以在列上移动一个元素时只需跨越一个存储单元。

华清远见元宇宙实验中心 xyzlab.com.cn

tensor的偏移

偏移是指从张量的第一个元素开始的索引位置。

```
a = torch.arange(12).reshape( *shape: 3, 4)
print(a)
# 使用偏移来选取第2行到第3行、第1列到第2列的元素
b = a[1:3, 0:2]
print(b)
```

选取第2行到第3行、第1列到第2列的元素，得到了一个新的张量b。

华清远见 HQYJ.COM
华清远见|元宇宙实验中心 yyzlab.com.cn

04 tensor的连续性

华清远见 HQYJ.COM
华清远见|元宇宙实验中心
yyzlab.com.cn

tensor的连续性是什么

华清远见 HQYJ.COM

Tensor的连续性指的是其元素在内存中按照其在张量中的顺序紧密存储，没有间隔或间隔是连续的。

```
a = torch.arange(12).reshape( *shape: 3, 4)
```

生成的tensor样子（张量中的顺序）：

0	1	2	3
4	5	6	7
8	9	10	11

展开后：

0	1	2	3	4	5	6	7	8	9	10	11

在storage存储（内存中）：

0	1	2	3	4	5	6	7	8	9	10	11

生成的tensor根据“行优先”策略，在展开后与storage存储的顺序是一致，即该tensor连续。

华清远见|元宇宙实验中心 yyzlab.com.cn

tensor的不连续性举例

```
a = torch.arange(12).reshape( *shape: 3, 4)
b = a.transpose(0, 1)
print(b)
print(b.flatten())
print(b.storage().tolist())
```

生成的tensor样子（张量中的顺序）：

0	4	8
1	5	9
2	6	10
3	7	11

展开后：

0	4	8	1	5	9	2	6	10	3	7	11

在storage存储（内存中）：

0	1	2	3	4	5	6	7	8	9	10	11

生成的tensor根据“行优先”策略，在展开后与storage存储的顺序是不一致，即该tensor不连续。

华清远见 HQYJ.COM
华清远见|元宇宙实验中心 yyzlab.com.cn

不连续的缺点与解决方案

华清远见 HQYJ.COM

当对Tensor进行某些操作，如转置（transpose）时，可能会导致Tensor变得不连续。

连续的tensor优势：

- 高效的内存访问：连续的张量在内存中占用一块连续的空间，这使得CPU可以高效地按顺序访问数据，减少了内存寻址的时间，从而提高了数据处理的速度。
- 优化的计算性能：在进行数学运算时，连续张量可以减少数据的移动和复制，因为数据已经按照计算所需的顺序排列，这样可以减少计算中的延迟，提高整体的计算性能。
- 其它：在不连续的tensor上进行view()操作会报错。

解决方案：

通过contiguous()方法将不连续的张量转换为连续的张量。

- 如果tensor不是连续的，则会重新开辟一块内存空间保证数据是在内存中是连续的。
- 如果Tensor 是连续的，则contiguous()无操作。

华清远见嵌入式学习实验中心 yyzlab.com.cn

华清远见
HQYJ.COM

科技赋能·智引未来

技术领先
品质保障

超多干货
实时更新

海量视频
贴身学习

了解更多信息请访问：

集团官网：www.hqyj.com

创客学院网址（在线课程）：www.makeru.com.cn

研发中心网址（研发产品）：www.fsdev.com.cn

元宇宙实验中心网址（在线实验）：www.yyzlab.com.cn

华清远见元宇宙实验中心
yyzlab.com.cn